



# 포팅 매뉴얼

## 0. 목차

1. 개요
  - a. 프로젝트 기술 (FE/BE/APP)
2. 빌드 & 실행
3. 배포 환경
4. DB
  - a. Azure MySQL 설정
  - b. 주요 설정
  - c. 스키마 구성
5. 외부 서비스 설정

## 1. 개요

### 프로젝트 기술

- FE

- React: 19.1.0
- TypeScript: 5.8.3
- Vite: 6.3.5
- Tailwind CSS: 3.4.17
- Zustand: 5.0.6
- React Router DOM: 7.7.1

- BE

- 프레임워크: Spring Boot (Java 17)
- 배포: Docker Compose (`/opt/app-backend`)

- API 엔드포인트: `/api/\*\*`
- CI/CD: Jenkins (dev 자동, main 수동)

- APP(Infra, Data)

- Nginx Reverse Proxy (HTTPS, SPA 라우팅, API 프록시)
- Jenkins (<https://j13a305.p.ssafy.io/jenkins>)
- MinIO (Object Storage → Spark 연계)
- Spark (B 서버 데이터 처리)

- APP (Android)

- UI: Jetpack Compose
- Architecture: MVVM + Repository 패턴
- 통신: Retrofit + CookieJar (세션 기반 인증)
- SDK: Samsung Health SDK (걸음 수, 수면 데이터 연동)
- 기타: FCM (푸시 알림), GPS, Google OAuth (Credential Manager)

## 2. 빌드 & 실행

### Front 빌드

```
cd frontend  
npm install  
npm run build
```

### Front 배포

```
sudo rm -rf /var/www/app/*  
sudo cp -r dist/* /var/www/app/
```

### Back 빌드

```
cd backend
./gradlew clean build -x test
docker build -t docker.io/dsazxc035/app-backend:${TAG} .
docker push docker.io/dsazxc035/app-backend:${TAG}
```

## Back 실행 (Api 서버)

```
cd /opt/app-backend
docker compose pull backend
docker compose up -d --force-recreate
docker logs app-backend --tail=100
```

## Spark 실행 (데이터 서버)

```
docker run -it --rm \
-v /opt/spark/jobs:/jobs \
-v /opt/spark/input:/input \
-v /opt/spark/logs:/logs \
bitnami/spark:3.5.1 /bin/bash

-----
# Spark Submit 예시
/opt/bitnami/spark/bin/spark-submit \
--packages org.apache.hadoop:hadoop-aws:3.3.6,com.amazonaws:aws-j
ava-sdk-bundle:1.12.767 \
--class org.example.JobMain \
/jobs/job.jar \
s3a://gold-bucket/gold/raw/bus_congestion/gold_bus_01.csv
```

## 3. 배포 환경

### 서버 A (서비스 서버)

- OS: Ubuntu 20.04+
- 오픈 포트: 22, 443
- 주요 서비스:

- Nginx (리버스 프록시, HTTPS)
- Jenkins (CI/CD)
- Docker (Backend, MinIO)

Nginx 주요 설정:

```
location / {
    try_files $uri $uri/ /index.html;
}

location /api/ {
    proxy_pass http://127.0.0.1:8081;
}

location /jenkins/ {
    proxy_pass http://127.0.0.1:8080/jenkins/;
}

location /minio/ {
    proxy_pass http://127.0.0.1:9000/;
}

location /minio-console/ {
    rewrite ^/minio-console(/.*)$ $1 break;
    proxy_pass http://127.0.0.1:9001/;
}
```

## 서버 B (데이터 서버)

- OS: Ubuntu 20.04+
- 오픈 포트: 22
- 주요 서비스:
  - Spark (Docker)
  - 데이터 업로드 경로: `/opt/spark/input/`
  - 작업 디렉토리: `/opt/spark/jobs/`

## 4. DB

### Azure MySQL 연결

```
mysql -h <DB_HOST> -u <DB_USER> -p --ssl-mode=REQUIRED
```

#### 주요 설정

- SSL 필수
- DB URL: `jdbc:mysql://<DB_HOST>:3306/<DB_NAME>?useSSL=true`
- max\_user\_connections 제한 확인:

```
SHOW VARIABLES LIKE 'max_user_connections';
```

### 스키마 구성

#### A) 사용자

##### 1) **member** — 사용자 기본

```
DROP TABLE IF EXISTS member;
CREATE TABLE member (
  id          BIGINT    NOT NULL AUTO_INCREMENT,
  google_key   VARCHAR(100) NOT NULL,
  email        VARCHAR(75) NOT NULL,
  nickname     VARCHAR(30) NOT NULL,
  role         ENUM('MEMBER','GUEST') NULL,
  health_permission BOOLEAN NOT NULL DEFAULT FALSE,
  push_permission BOOLEAN NOT NULL DEFAULT FALSE,
  sleep_permission BOOLEAN NOT NULL DEFAULT FALSE,
  fatigue      INT       NOT NULL DEFAULT 0,
  energy       INT       NOT NULL DEFAULT 50,
  foot_step    INT       NOT NULL DEFAULT 0,
  created_at   DATETIME(6) NOT NULL DEFAULT CURRENT_TIMESTAMP
(6),
  updated_at   DATETIME(6) NULL,
```

```

is_withdraw    BOOLEAN    NOT NULL DEFAULT FALSE,
PRIMARY KEY (id),
UNIQUE KEY uq_member_google (google_key),
UNIQUE KEY uq_member_email (email)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_0900_
ai_ci
COMMENT='사용자';

```

## B) 건강 · 피로

### 2) **health\_info\_log** — 건강 스냅샷 로그

```

DROP TABLE IF EXISTS health_info_log;
CREATE TABLE health_info_log (
  id      BIGINT      NOT NULL AUTO_INCREMENT,
  created_at DATETIME(6) NOT NULL,
  foot_step INT       NULL,
  member_id BIGINT     NOT NULL,
  stress  INT         NULL,
  PRIMARY KEY (id),
  KEY idx_hil_member_created (member_id, created_at)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_0900_
ai_ci
COMMENT='건강 상태 원본 로그';

```

### 3) **health\_info\_daily** — 일 단위 집계

```

DROP TABLE IF EXISTS health_info_daily;
CREATE TABLE health_info_daily (
  id      BIGINT      NOT NULL AUTO_INCREMENT,
  created_at DATETIME(6) NOT NULL, -- KST 자정 고정 권장
  foot_step INT       NULL,
  member_id BIGINT     NOT NULL,
  stress  INT         NULL,
  PRIMARY KEY (id),

```

```

    UNIQUE KEY uq_hid_member_day (member_id, created_at),
    KEY idx_hid_member (member_id)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_0900_
ai_ci
COMMENT='일평균 건강 상태(일 단위)';

```

#### 4) **fatigue\_log** — 피로도 이벤트

```

DROP TABLE IF EXISTS fatigue_log;
CREATE TABLE fatigue_log (
    id          BIGINT      NOT NULL AUTO_INCREMENT,
    created_at   DATETIME(6) NOT NULL,
    fatigue      INT         NULL,
    fatigue_change INT       NULL,
    member_id    BIGINT      NOT NULL,
    reason       ENUM('COFFEE','NAP','TRAFFIC','WALK') NULL,
    PRIMARY KEY (id),
    KEY idx_fl_member_created (member_id, created_at)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_0900_
ai_ci
COMMENT='피로도 이벤트 로그';

```

### C) 경로 · 이용 로그

#### 5) **route\_info** — 경로 상세 스텝(복합 PK)

```

DROP TABLE IF EXISTS route_info;
CREATE TABLE route_info (
    `order`      INT         NOT NULL,
    route_id     BIGINT      NOT NULL,
    created_at    DATETIME(6) NOT NULL,
    departure     VARCHAR(255) NULL,
    destination   VARCHAR(255) NULL,
    line_name     VARCHAR(255) NULL,
    time_taken    INT         NULL,

```

```

`type`      VARCHAR(16) NULL,  -- BUS/SUBWAY/WALK 등
walk_distance INT      NULL,
end_lat     DOUBLE     NULL,
end_lng     DOUBLE     NULL,
start_lat   DOUBLE     NULL,
start_lng   DOUBLE     NULL,
bus_route_id BIGINT     NULL,
bus_station_id BIGINT   NULL,
path_json   TEXT       NULL,
PRIMARY KEY (`order`, route_id),
KEY idx_ri_route (route_id),
KEY idx_ri_type (`type`)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_0900_
ai_ci
COMMENT='경로 상세 스텝';

```

## 6) **route\_log** — 경로 선택/예측 기록

```

DROP TABLE IF EXISTS route_log;
CREATE TABLE route_log (
  id          BIGINT      NOT NULL AUTO_INCREMENT,
  created_at   DATETIME(6) NOT NULL,
  departure    VARCHAR(255) NULL,
  destination  VARCHAR(255) NULL,
  fatigue      INT        NULL,
  predicted_fatigue INT     NULL,
  predicted_time INT       NULL,
  started_at   DATETIME(6) NULL,
  time_taken   INT        NULL,
  used         BIT(1)     NULL,
  member_id    INT        NULL,
  PRIMARY KEY (id),
  KEY idx_rl_member_created (member_id, created_at)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_0900_
ai_ci

```

```
COMMENT='경로 이용/예측 로그';
```

## D) 즐겨찾기

### 7) **bookmark\_route** — 즐겨찾기 루트(헤더)

```
DROP TABLE IF EXISTS bookmark_route;
CREATE TABLE bookmark_route (
  id          BIGINT    NOT NULL AUTO_INCREMENT,
  created_at   DATETIME(6) NOT NULL,
  departure    VARCHAR(255) NULL,
  departure_latitude DOUBLE NULL,
  departure_longitude DOUBLE NULL,
  departure_name VARCHAR(255) NULL,
  destination  VARCHAR(255) NULL,
  destination_latitude DOUBLE NULL,
  destination_longitude DOUBLE NULL,
  destination_name VARCHAR(255) NULL,
  `name`       VARCHAR(255) NULL,    -- 즐겨찾기 별칭
  `type`       ENUM('BUS','SUBWAY','WALKING') NULL,
  member_id    INT      NULL,
  PRIMARY KEY (id),
  KEY idx_br_member_created (member_id, created_at)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_0900_ai_ci
COMMENT='즐거찾기 루트(헤더)';
```

### 8) **bookmark\_route\_info** — 즐겨찾기 스텝(디테일)

```
DROP TABLE IF EXISTS bookmark_route_info;
CREATE TABLE bookmark_route_info (
  id          BIGINT    NOT NULL AUTO_INCREMENT,
  `order`     INT      NULL,
  created_at   DATETIME(6) NOT NULL,
  departure_name VARCHAR(255) NULL,
```

```

destination_name VARCHAR(255) NULL,
end_lat          DOUBLE      NULL,
end_lng          DOUBLE      NULL,
line_name        VARCHAR(255) NULL,
start_lat        DOUBLE      NULL,
start_lng        DOUBLE      NULL,
`type`           ENUM('BUS','SUBWAY','WALKING') NULL,
walk_distance    INT          NULL,
bookmark_id      BIGINT       NULL,      -- 논리 FK: bookmark_route.id
time_taken       INT          NULL,
bus_route_id     VARCHAR(255) NULL,
bus_station_id   VARCHAR(255) NULL,
path_json        TEXT         NULL,
PRIMARY KEY (id),
KEY idx_bri_bookmark_order (bookmark_id, `order`)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_0900_
ai_ci
COMMENT='즐거찾기 루트 스텝';

```

운영에서 FK를 강제하고 싶으면 아래처럼 선택 적용:

```

-- ALTER TABLE bookmark_route_info
-- ADD CONSTRAINT fk_bri_bookmark
-- FOREIGN KEY (bookmark_id) REFERENCES bookmark_route(id)
-- ON DELETE CASCADE;

```

## E) 골드 레이어(데이터 마트)

### 9) **gold\_transfer** — 환승 피로도 산출 결과

```

DROP TABLE IF EXISTS gold_transfer;
CREATE TABLE gold_transfer (
  id          INT          NOT NULL AUTO_INCREMENT,
  line_no     VARCHAR(10)  NULL,
  station     VARCHAR(80)  NULL,

```

```

latitude      DECIMAL(10,7) NULL,
longitude      DECIMAL(10,7) NULL,
distance_m     INT          NULL,
duration_min   DECIMAL(10,6) NULL,
passengers_avg DECIMAL(12,3) NULL,
distance_norm  DECIMAL(10,6) NULL,
duration_norm  DECIMAL(10,6) NULL,
passengers_norm DECIMAL(10,6) NULL,
fatigue_score  DECIMAL(10,6) NULL,
fatigue_rank   INT          NULL,
fatigue_percentile DECIMAL(10,6) NULL,
`line`         INT          NULL,
`distance`     INT          NULL,
PRIMARY KEY (id),
KEY idx_gt_station (station),
KEY idx_gt_line_no (line_no),
KEY idx_gt_line (`line`)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_0900_
ai_ci
COMMENT='환승 피로도 골드 레이어';

```

CSV 적재 예시:

```

-- LOAD DATA INFILE '/var/lib/mysql-files/gold_transfer.csv'
-- INTO TABLE gold_transfer
-- FIELDS TERMINATED BY ',' ENCLOSED BY '"'
-- LINES TERMINATED BY '\n'
-- IGNORE 1 ROWS;

```

## 5. 외부 서비스 설정

| 모든 키는 환경변수 또는 .env로 주입

**ODSay**

```
odsay.api.key=${ODSAY_API_KEY}
```

## Naver (Search / Geocoding/ MAPs)

```
naver.api.search.client-id=${NAVER_SEARCH_CLIENT_ID}  
naver.api.search.client-secret=${NAVER_SEARCH_CLIENT_SECRET}  
naver.api.geocoding.client-id=${NAVER_GEOCODING_CLIENT_ID}  
naver.api.geocoding.client-secret=${NAVER_GEOCODING_CLIENT_SECRET}
```

## DB - My SQL

```
spring.datasource.driver-class-name=com.mysql.cj.jdbc.Driver  
spring.datasource.url=${DB_URL}  
spring.datasource.username=${DB_USERNAME}  
spring.datasource.password=${DB_PASSWORD}  
spring.jpa.hibernate.ddl-auto=update  
spring.jpa.hibernate.naming.physical-strategy=org.hibernate.boot.model.naming.PhysicalNamingStrategyStandardImpl
```

## Google OAuth2 (로그인)

```
spring.security.oauth2.client.registration.google.client-name=google  
spring.security.oauth2.client.registration.google.client-id=${GOOGLE_CLIENT_ID}  
spring.security.oauth2.client.registration.google.client-secret=${GOOGLE_CLIENT_SECRET}  
spring.security.oauth2.client.registration.google.redirect-uri=${GOOGLE_REDIRECT_URI}  
spring.security.oauth2.client.registration.google.authorization-grant-type=authorization_code  
spring.security.oauth2.client.registration.google.scope=profile,email,openid  
  
# provider (Google 기본값 사용 시 생략 가능 / 커스텀 필요 시 채워넣기)  
spring.security.oauth2.client.provider.google.authorization-uri=https://acco
```

```
unts.google.com/o/oauth2/v2/auth
spring.security.oauth2.client.provider.google.token-uri=https://oauth2.goo
gleapis.com/token
spring.security.oauth2.client.provider.google.user-info-uri=https://www.go
ogleapis.com/oauth2/v3/userinfo
spring.security.oauth2.client.provider.google.user-name-attribute=sub
```

## Redis

```
spring.data.redis.host=localhost
spring.data.redis.port=6379
spring.data.redis.timeout=3000
spring.data.redis.ssl.enabled=false
```

## Google Weather

```
weather.google.base-url=https://weather.googleapis.com/v1/currentCondi
tions:lookup
weather.google.api-key=${GOOGLE_WEATHER_ACCESS_KEY}
```

## AWS (선택: S3 사용 시)

```
AWS_ACCESS_KEY_ID=${AWS_ACCESS_KEY_ID}
AWS_SECRET_ACCESS_KEY=${AWS_SECRET_ACCESS_KEY}
AWS_REGION=${AWS_REGION:ap-northeast-2}
S3_BUCKET=${S3_BUCKET}
```

## MinIO

컨테이너 실행:

```
cd /opt/minio
docker run -d --name minio \
-p 9000:9000 -p 9001:9001 \
-v /opt/minio/data:/data \
-e "MINIO_ROOT_USER=admin" \
```

```
-e "MINIO_ROOT_PASSWORD=admin123" \
quay.io/minio/minio server /data --console-address ":9001"
```

버킷 생성:

```
mc alias set s3w https://j13a305.p.ssafy.io/minio <ACCESS_KEY> <SECRET_KEY> --api S3v4
mc mb s3w/gold-bucket
```

## Android

- Google Cloud Console 클라이언트 auth

```
WEB_CLIENT_ID=81808688241-346b329me48o028eqjs979u2st77c9n0.ap
ps.googleusercontent.com
ANDROID_CLIENT_ID=81808688241-07o84c83eicaf98em0fv2p0ahn4ed88
l.apps.googleusercontent.com
```

## Jenkins

- GitLab Webhook으로 트리거
- Credentials:
  - `gitlab_https` (GitLab Deploy Token)
  - `dockerhub_registry` (Docker Hub)
- Frontend Pipeline: dev 브랜치 푸시 시 자동 배포
- Backend Pipeline: dev 자동 빌드 → 배포, main 수동 배포